Práctica N°4: Sistemas RTOS

Índice

1. Objetivos **2**
- 1.1. Objetivo general . . . 2
- 1.2. Objetivos específicos . . . 2
- 1.3. Asignación de Pines en el STM32CubeMX . . . 2
  - 1.3.1. Pines para los LEDs Integrados . . . 3
  - 1.3.2. Pin para el Pulsador de Usuario . . . 4
  - 1.3.3. Pines para el UART2 . . . 4
- 1.4. Configuración de FreeRTOS en CubeMX . . . 5
  - 1.4.1. Tareas definidas en el código . . . 5

2. Resultados **6**
- 2.1. Comprobación de la ejecución de tareas . . . 6
- 2.2. Comunicación por UART2 . . . 8

1. Objetivos

El propósito de esta práctica es introducir al estudiante en el uso de un sistema operativo en tiempo real (RTOS) aplicado al microcontrolador STM32F407, utilizando el entorno de desarrollo STM32CubeIDE y el kernel FreeRTOS. Se busca comprender cómo se gestionan las tareas concurrentes, las prioridades, los retardos y la sincronización mediante interrupciones y semáforos.

1.1. Objetivo general

Implementar un sistema multitarea basado en FreeRTOS sobre el microcontrolador STM32F407G-DISC1, que permita ejecutar tareas con distintas prioridades y sincronizar su funcionamiento a través de interrupciones externas, utilizando los mecanismos de planificación y temporización del RTOS.

1.2. Objetivos específicos

- Comprender la estructura básica de un proyecto con FreeRTOS generado mediante STM32CubeIDE.
- Configurar correctamente el kernel de FreeRTOS dentro del entorno STM32CubeMX, habilitando las tareas, prioridades y temporizadores del sistema.
- Implementar varias tareas concurrentes con diferentes niveles de prioridad y analizar su comportamiento en ejecución.
- Utilizar la comunicación mediante interrupciones externas para modificar el flujo de ejecución de una o más tareas.
- Observar el efecto de los retardos (`osDelay()`) y la planificación por prioridad sobre el tiempo de ejecución de cada tarea.
- Consolidar el uso del puerto serie (UART) para la depuración y monitoreo del sistema multitarea.

1.3. Asignación de Pines en el STM32CubeMX

Una parte fundamental en la configuración de un proyecto en STM32CubeIDE es la asignación de pines dentro del archivo `.ioc`. Cada periférico del microcontrolador debe conectarse correctamente a los pines físicos de la tarjeta de desarrollo.

En esta práctica se requiere configurar los periféricos del **UART2**, los **LEDs integrados** y el **pulsador de usuario**.

1.3.1. Pines para los LEDs Integrados

La tarjeta STM32F407G-DISC1 dispone de cuatro LEDs de usuario conectados al puerto D del microcontrolador. Cada LED puede ser controlado individualmente mediante escritura digital sobre los pines correspondientes:

- **PD12**: LED Verde (LD4)
- **PD13**: LED Naranja (LD3)
- **PD14**: LED Rojo (LD5)
- **PD15**: LED Azul (LD6)

Cada uno de estos pines debe configurarse como **GPIO Output** en el archivo `.ioc`, con los siguientes parámetros:

- Mode: Output Push-Pull.
- Pull-up/Pull-down: No pull.
- Maximum Output Speed: High.

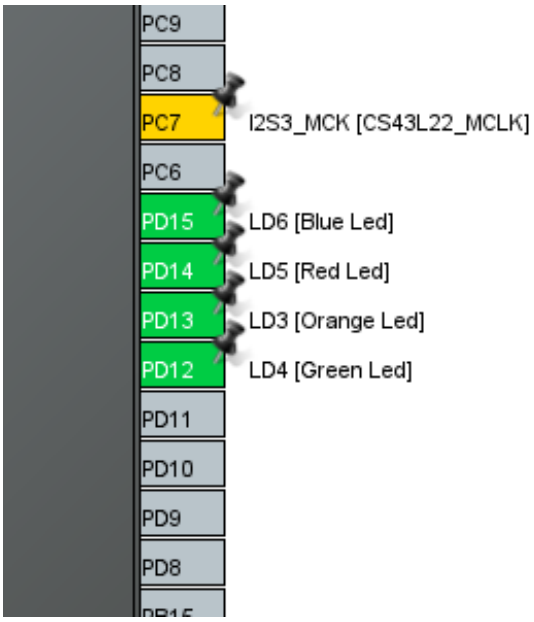


Figura 1: Configuración de pines PD12–PD15 para los LEDs integrados del STM32F407G-DISC1.

1.3.2. Pin para el Pulsador de Usuario

El pulsador de usuario en la placa (USER BUTTON) se encuentra conectado al pin **PA0**. Este se configura como una **Interrupción externa (EXTI0)** que permite detectar los flancos de subida cuando el botón es presionado.

- **PA0**: GPIO External Interrupt, Rising Edge.

Además, es necesario habilitar en la pestaña **NVIC** la interrupción correspondiente a **EXTI0** para que pueda ser manejada desde el código generado.

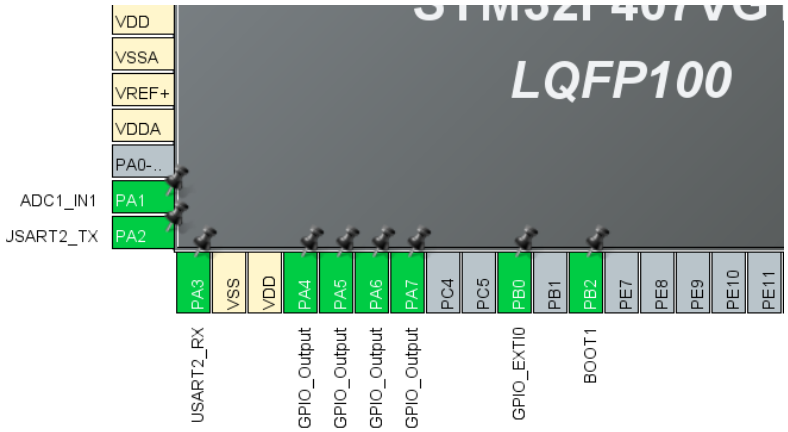


Figura 2: Configuración del pin PA0 como interrupción externa para el pulsador de usuario.

1.3.3. Pines para el UART2

El periférico **USART2** se utiliza para la comunicación serie con la PC, permitiendo la visualización de mensajes y depuración a través del puerto virtual (VCP). Los pines asignados por defecto en la tarjeta STM32F407G-DISC1 son:

- **PA2**: USART2_TX (Transmisión)
- **PA3**: USART2_RX (Recepción)

Configuración recomendada para el UART2:

- Baud Rate: 115200 bps.
- Word Length: 8 Bits.

- Stop Bits: 1.
- Parity: None.
- Mode: TX/RX.

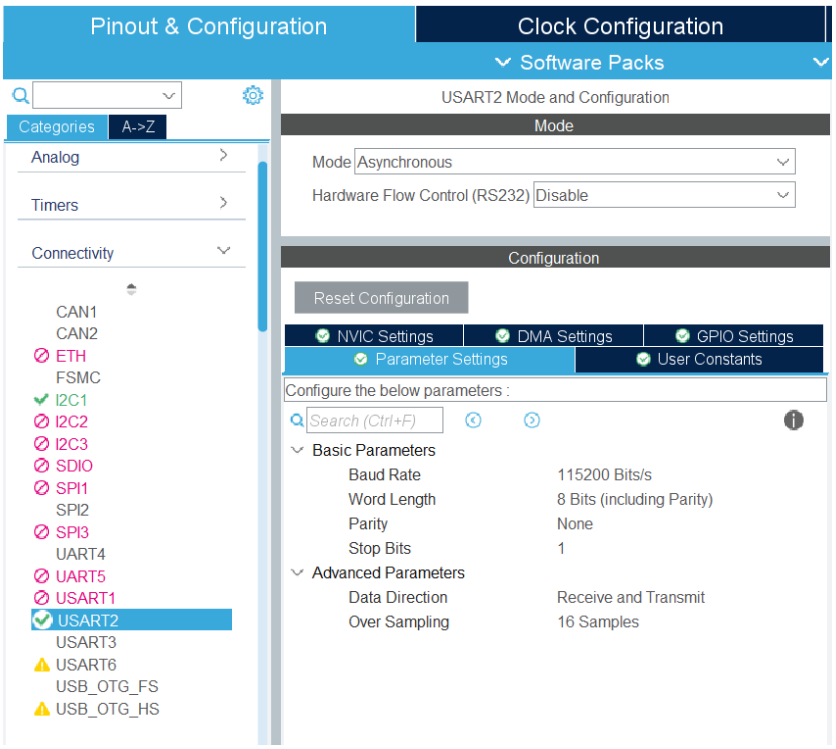


Figura 3: Configuración del periférico UART2 en CubeMX.

1.4. Configuración de FreeRTOS en CubeMX

Para gestionar tareas concurrentes dentro del proyecto, se activa el sistema operativo en tiempo real **FreeRTOS**. Esta configuración se realiza en la pestaña **Middleware ¿FREERTOS**.

1.4.1. Tareas definidas en el código

- **defaultTask** — Handler: `StartDefaultTask` — **Prioridad: osPriorityAboveNormal**. Descripción: tarea de prioridad "media"que alterna el LED naranja (PD13) y emite por UART el mensaje "Tarea Media ejecutadaçada 500 ms.

- **TareaBaja** — Handler: `StartTareaBaja` — **Prioridad: osPriorityLow**. Descripción: tarea de baja prioridad que alterna el LED verde (PD12) cada 1000 ms.
- **TareaAlta** — Handler: StartTareaAlta — Prioridad: osPriorityHigh. Descripción: tarea de alta prioridad que controla el LED rojo (PD14) en función de la variable global volátil `led_rojo_activado`. Si `led_rojo_activado` es 1, enciende el LED durante 2000 ms; si es 0 apaga el LED y espera 500 ms.

En el ioc se generan las tareas, sin embargo su funcionalidad se genera en el código por lo que de manera visual solo se crea el espacio para su posterior programación.

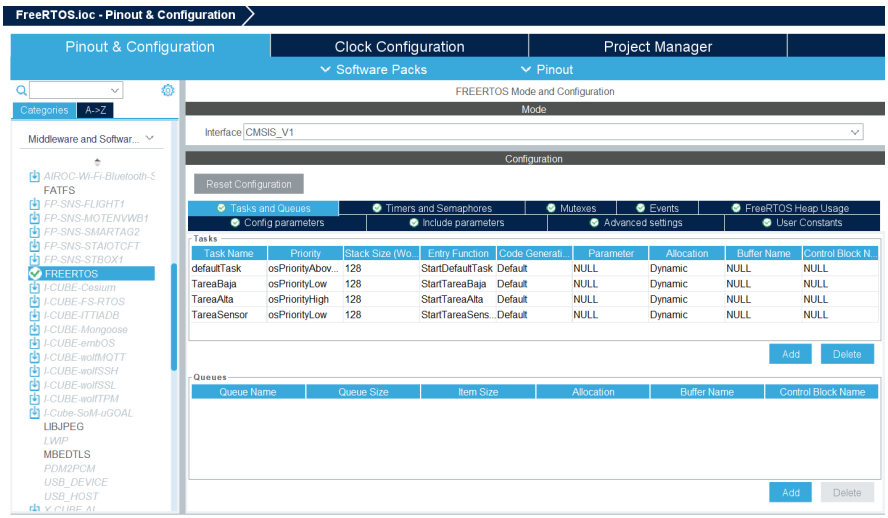


Figura 4: Configuración de tareas y prioridades en FreeRTOS desde STM32CubeMX.

2. Resultados

Una vez configurados los pines, periféricos y tareas en FreeRTOS, se procedió a cargar el programa en la placa de desarrollo **STM32F407G-DISC1**. El objetivo de esta fase fue comprobar el funcionamiento concurrente de las tareas, la correcta lectura del pulsador y la transmisión de datos por UART.

2.1. Comprobación de la ejecución de tareas

Durante la ejecución se observó que las tres tareas del sistema (alta, baja y por defecto) operan de forma simultánea bajo la planificación de FreeRTOS:

- La tarea **TareaAlta** (prioridad alta) controla el LED rojo (PD14) según el estado de la variable global modificada en la interrupción externa.
- La tarea **defaultTask** (prioridad normal) realiza el parpadeo del LED naranja (PD13) y envía mensajes periódicos por el puerto UART2.
- La tarea **TareaBaja** (prioridad baja) controla el LED verde (PD12) con una frecuencia de parpadeo más lenta.

En la siguiente figura se observa la placa durante la ejecución, con los tres LEDs funcionando de manera independiente.

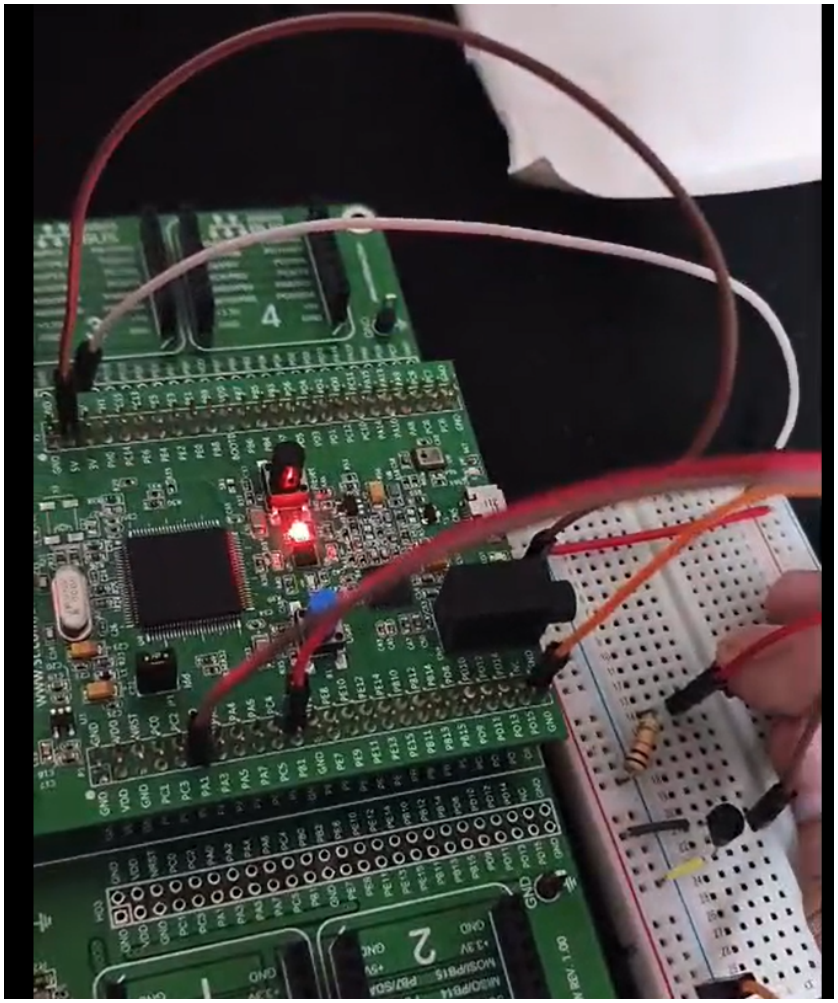


Figura 5: Ejecución de las tareas: LEDs parpadeando de manera concurrente bajo FreeRTOS.

2.2. Comunicación por UART2

La comunicación serial a través de **USART2** permitió verificar el comportamiento de las tareas mediante mensajes enviados al monitor serie (velocidad de 115200 baudios). Cada 500 ms, la tarea principal imprime el estado del sistema, demostrando que FreeRTOS mantiene una temporización estable.

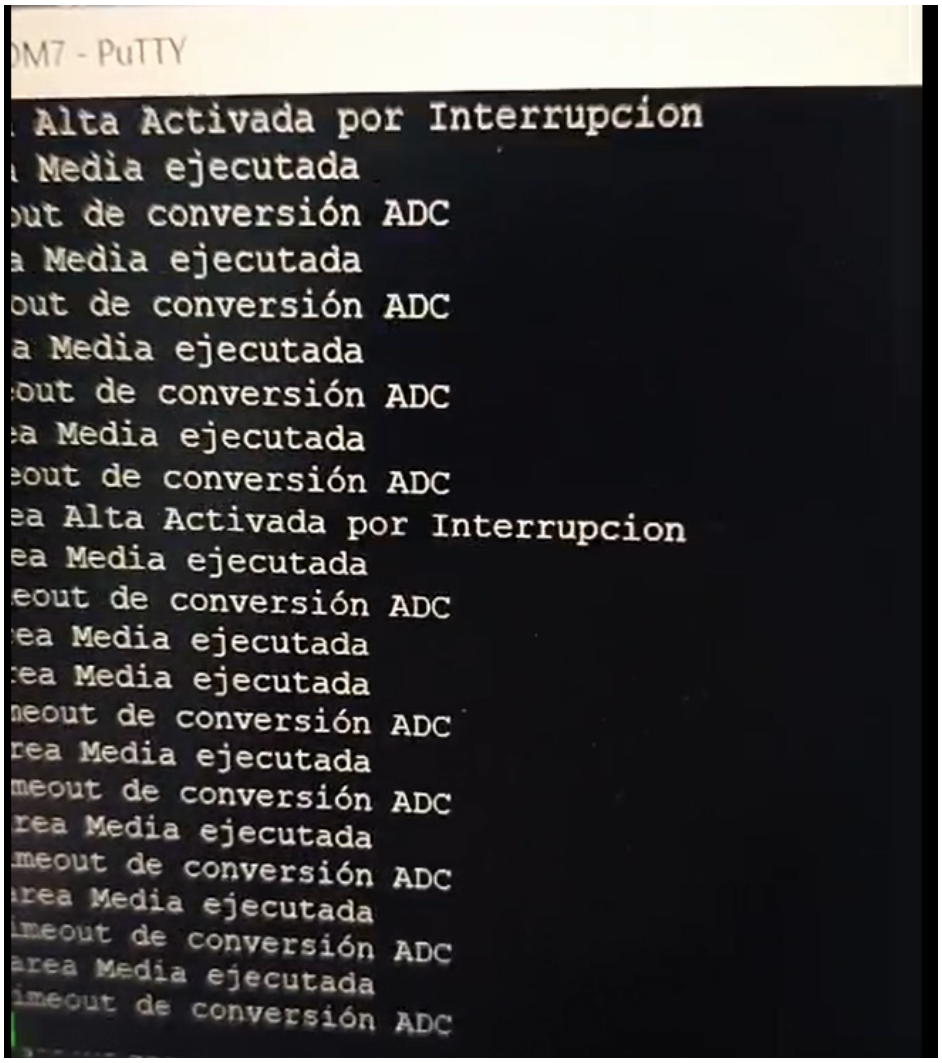


Figura 6: Mensajes enviados por UART2 durante la ejecución del programa.